

Open Datacenter Modernization

From virtualization estate to an open, hybrid application platform

A practical decision framework for platform strategy, workload placement, migration, economics, skills and operations

Architecture position

**Let workload, operating-model and economic decisions
shape the target datacenter - not the other way around.**

Decision journey

Why

Assess

Platform

Place

Consume

Migrate

Operate

Sumit Mukherjee

Architecture Point of View / September 2026 / v1.1

Independent professional architecture content

Executive premise

Modernize the datacenter, not just the hypervisor.

The target datacenter should emerge from workload, operating-model and economic decisions - not from a predetermined technology destination.

1 Why	2 Assess	3 Platform	4 Place	5 Consume	6 Migrate	7 Operate
----------	-------------	---------------	------------	--------------	--------------	--------------

Executive premise

Datacenter modernization should not begin with the question, “What should replace my current hypervisor?”

That question assumes the future datacenter should preserve the architecture and operating model of the present one. Sometimes that is exactly the right answer. Often it is not.

A more useful starting point is:

What application and infrastructure operating model does the organization need for the next 5-10 years, and which workloads should run on virtualization, Kubernetes, a unified VM-and-container platform, bare metal, public cloud or SaaS?

A hypervisor replacement, Kubernetes adoption, hardware refresh, cloud migration and datacenter consolidation can all be parts of modernization, but none of them is synonymous with modernization.

The objective is to make deliberate decisions about workload disposition, execution model, placement, consumption model and operating model before selecting products.

Core principles

- Modernize by workload, not by platform mandate.
- Migration and application modernization do not have to happen at the same time.
- A hypervisor replacement can be valid, but it should be recognized as a virtualization decision rather than automatically called datacenter modernization.
- Kubernetes is an application platform and operating model, not a requirement for every workload.

- A unified VM-and-container platform can reduce operational silos, but convergence has value only when the organization can operate it effectively.
- Public cloud is a placement and consumption option, not a universal destination.
- Build, buy and consume are business operating-model choices as much as technology choices.
- The target architecture is not complete until Day-2 operations, resilience, security, skills and economics are understood.



1. Establish the modernization trigger

The phrase “modernize the datacenter” hides very different business problems. The first workshop should identify the actual trigger, the deadline, the economic pressure and the outcome expected.

Trigger	Immediate question	Strategic question
Virtualization cost or licensing	What alternatives exist?	Should the future platform remain VM-centric?
Hardware/storage refresh	What should be refreshed?	Which workloads should still be on-premises?
Datacenter consolidation	What moves where?	Can the operating model be simplified at the same time?
Kubernetes demand	Which platform?	Which workloads benefit from a cloud-native operating model?
Application modernization	Which applications change?	Can migration and modernization be decoupled?
Cloud mandate	What can move?	Which workloads actually benefit from cloud economics and services?
Resilience/DR gaps	How do we improve recovery?	Does the target platform change the resilience model?
Skills/operations pressure	How do we reduce toil?	What should be self-operated versus supported or managed?
Security/compliance	What controls are missing?	Can policy, identity and audit become platform capabilities?
AI/accelerated compute	Where do new workloads run?	How should accelerators coexist with the existing estate?

Executive output

Document a one-sentence modernization mandate:

We are modernizing because [trigger], by [time constraint], to achieve [measurable business/operational outcome], while preserving [critical constraints].

If that sentence cannot be agreed, product selection is premature.

2. Assess before you architect

Inventory is necessary, but inventory alone is not assessment. A modernization program needs to understand what runs, why it exists, what it depends on, what it costs, how critical it is and how difficult it is to change.

2.1 Workload discovery dataset

For each application/workload capture:

- Business owner and technical owner
- Business criticality
- Environment: production / DR / test / development
- OS and support lifecycle
- VM, container, bare-metal or appliance form
- Allocated and observed CPU/memory
- Storage capacity, growth, latency and IOPS characteristics
- Network segments, load balancers, firewall rules and external dependencies
- Application and database dependencies
- Availability requirement
- RPO / RTO
- Backup and restore requirements
- Security/data classification
- Licensing constraints
- Hardware dependencies, GPUs, USB/dongles, special devices
- Maintenance windows and acceptable downtime
- Current operational owner and support model
- Current run cost where available
- Modernization appetite and business roadmap

2.2 Assess utilization, not only allocation

Do not size the target estate by copying current VM allocations. Existing environments often contain over-provisioning, abandoned workloads and historical sizing assumptions.

A target capacity model should consider:

Observed demand + growth + HA headroom + failure-domain capacity + operational reserve + migration coexistence

rather than simply:

Current allocated vCPU + current allocated RAM

2.3 Identify dependencies before migration waves

A VM is rarely the real unit of migration. Applications depend on databases, identity, DNS, certificates, file shares, message brokers, IP addresses, load balancers, security rules and other applications.

Create an application dependency map before assigning migration waves.



3. Decide workload disposition

Before choosing where a workload should run, decide what should happen to it.

Disposition	Meaning	Typical use
Retain	Leave where it is for now	Stable workload with no compelling event
Rehost	Move substantially unchanged	VM migration / infrastructure exit
Replatform	Change platform with limited application change	VM-to-container where practical; managed database adoption
Refactor	Change application architecture	Cloud-native decomposition, resilience or scale requirement
Replace	Adopt SaaS/package	Commodity capability or obsolete custom application
Retire	Decommission	Duplicate, unused or end-of-life workload

Decision rule

Do not spend modernization budget transforming a workload whose business future is uncertain.

Retirement and replacement decisions can remove more infrastructure cost than technical optimization.

4. Choose the execution model: virtualization, Kubernetes or both

This is the pivotal architecture decision.

The useful question is not “VMs or containers?” It is:

Which execution model best fits each workload, and should the enterprise operate those models as separate silos or through a common platform?

4.1 Virtualization-centric platform

Best fit when:

- the estate remains predominantly VM-based;
- application change is limited;
- existing operational skills are strongly virtualization-oriented;
- the immediate objective is infrastructure replacement or cost control;
- application-platform convergence provides little near-term value.

Strengths:

- lower transformation effort;
- familiar VM lifecycle;
- straightforward rehosting.

Trade-offs:

- may preserve VM-centric provisioning and operational silos;
- container/application platform remains a separate concern;
- may solve the current commercial event without changing the long-term operating model.

4.2 Kubernetes-centric platform

Best fit when:

- new applications are predominantly containerized;
- platform engineering and developer self-service are strategic;
- applications are designed for cloud-native lifecycle and automation;
- VM workloads are small, declining or handled elsewhere.

Strengths:

- declarative automation;

- application-centric platform services;
- strong fit for containers and modern delivery practices.

Trade-offs:

- existing VM estate still needs an answer;
- platform skills and Day-2 maturity are prerequisites;
- forcing unsuitable applications into containers can increase risk and cost.

4.3 Unified VM + container platform

Best fit when:

- a significant VM estate must move;
- application modernization will happen over several years;
- the organization wants common automation, policy, identity, networking and operations;
- reducing separate virtualization and Kubernetes silos has meaningful value.

The strategic benefit is decoupling migration from modernization:

Day 1: rehost a VM onto the new platform. Later: replatform or refactor selected applications when business value justifies it.

Trade-offs:

- the platform may be operationally more complex than a hypervisor-only environment;
- teams need Kubernetes/platform engineering skills even for VM-centric operations;
- feature parity should be evaluated against actual workload requirements, not assumed.

4.4 Mixed platform estate

A valid target can combine:

- traditional virtualization;
- unified VM/container platform;
- bare metal;
- public cloud;
- edge platforms;
- SaaS.

Architectural consistency does not require every workload to use the same execution substrate.



5. Workload placement framework

Use a sequence of decisions rather than a product-first destination.

Step 1 - Does the workload still need to exist?

If no: Retire. If a SaaS/package provides a better business outcome: Replace.

Step 2 - Is application change justified?

If no: favor rehost / VM. If limited change is justified: consider replatform. If material business value justifies architecture change: consider refactor.

Step 3 - Select execution model

Choose among:

- VM
- container
- bare metal
- SaaS/serverless/managed application service where applicable

Evaluate:

- OS/application support
- statefulness
- hardware dependencies
- licensing
- performance/latency
- availability model
- operational maturity
- modernization roadmap

Step 4 - Select placement

Choose:

- private datacenter
- colocation
- public cloud
- edge
- hybrid combination

Evaluate:

- demand variability
- latency
- data gravity
- sovereignty/compliance
- connectivity
- existing investment
- cloud egress
- resilience
- service availability
- economics

Step 5 - Select consumption model

Choose:

- DIY / self-operated open source
- enterprise-supported platform
- managed service / cloud service

Evaluate:

- control
- skills
- time to value
- supportability
- customization
- operational responsibility
- lock-in/coupling
- cost predictability

6. Private, public or hybrid?

“Cloud first” and “on-premises first” are both poor substitutes for workload analysis.

Private/on-premises tends to be attractive when

- workloads are predictable and continuously utilized;
- data gravity or latency is significant;
- sovereignty or isolation requirements are strong;
- existing facilities and infrastructure are economically useful;
- specialized hardware must remain local;
- the organization has strong platform operations.

Public cloud tends to be attractive when

- demand is variable or temporary;
- rapid geographic expansion matters;
- managed services create substantial engineering leverage;
- environments are short-lived;
- experimentation speed matters;
- the organization deliberately prefers consumption over infrastructure ownership.

Hybrid is often the natural result

A modern platform should make placement a deliberate decision while minimizing unnecessary differences in application delivery, security and operations.

Cloud is not automatically cheaper; on-premises is not automatically more controlled. Both outcomes depend on architecture and operating discipline.

7. Build, buy or consume?

A modernization program must decide not only what technology to use, but who owns the integration and operational responsibility.

Dimension	DIY / Build	Supported platform / Buy	Managed service / Consume
Control	Highest	High	Lower
Customization	Highest	High	Constrained by service
Internal skills needed	Highest	Medium-high	Lower
Integration responsibility	Enterprise	Shared	Provider-heavy
Operational responsibility	Enterprise	Enterprise with vendor support	Provider-heavy
Time to value	Usually longer	Medium	Usually faster
Upgrade/lifecycle burden	Enterprise	Shared guidance/support	Provider
Cost model	People + infra + integration	Subscription + people + infra	Consumption + service
Coupling	Project/skills coupling	Vendor/platform coupling	Service/API coupling

Decision principle

Own what differentiates the business; buy or consume what creates little differentiation but significant operational burden.

Most large enterprises will intentionally use all three models.

8. Define the target platform capabilities

Only after workload, execution, placement and consumption decisions should the target architecture be specified.

Application execution

- Virtual machines
- Containers
- Bare-metal/specialized workloads where needed

Platform services

- Identity and access
- Network and load balancing
- Storage and software-defined storage where appropriate
- Registry/artifact services
- Secrets/certificate management
- Policy and admission controls
- Observability
- Backup and recovery
- Disaster recovery
- Automation / IaC / GitOps
- Developer self-service
- Cost/capacity visibility

Infrastructure

- Compute
- Network
- block/file/object storage
- accelerators
- facilities / cloud infrastructure

Cross-cutting requirements

- availability
- security
- compliance



- lifecycle
- supportability
- interoperability
- automation
- operational ownership

The vendor-neutral capability architecture should be approved before creating the product mapping.



9. Migration factory

Large migrations should be treated as a repeatable production system.

Discover -> Assess -> Classify -> Dependency map -> Target placement -> Remediate -> Wave plan
-> Migrate -> Validate -> Cut over -> Optimize -> Decommission

9.1 Migration waves

A wave should group workloads according to:

- dependency boundaries;
- business windows;
- common migration pattern;
- target platform readiness;
- testing ownership;
- rollback feasibility.

Do not group solely by VM count.

9.2 Pilot strategy

Use pilots to validate unknowns, not to prove that the easiest VM can boot.

A good pilot portfolio includes:

- simple stateless VM;
- stateful application;
- representative network/security pattern;
- representative storage pattern;
- backup/restore;
- DR or recovery test where in scope;
- operational monitoring;
- one application with meaningful dependencies.

9.3 Exit criteria

A migrated workload is not complete when it starts on the target platform.

Completion should include:

- application validation;
- performance acceptance;
- backup protection;
- monitoring;



- security controls;
- support handover;
- rollback window closure;
- source decommission plan;
- CMDB/inventory update;
- cost ownership.



10. Estimate migration complexity and effort

Do not estimate by VM count alone.

Score each application/workload group from 1 (low) to 5 (high) across:

- application complexity;
- dependency complexity;
- data volume/change rate;
- downtime sensitivity;
- network/security complexity;
- compliance criticality;
- platform transformation required;
- testing difficulty;
- owner/readiness uncertainty.

Suggested interpretation

- 9-17: Low complexity
- 18-27: Medium complexity
- 28-36: High complexity
- 37-45: Very high complexity / architecture review required

These bands are planning heuristics, not industry benchmarks.

Effort model

For a migration wave:

Wave effort = discovery/remediation + platform preparation + migration execution + testing + cutover + stabilization

Program duration is influenced by:

- discovery completeness;
- remediation backlog;
- target platform readiness;
- business blackout windows;
- dependency sequencing;
- migration tooling;
- data transfer;
- test capacity;
- number of parallel teams;

- approval/change processes.

A useful throughput model is:

Monthly migration throughput = eligible workloads per wave x waves per month x success factor

The “success factor” accounts for remediation, failed validation and rescheduling. It should be derived from pilot/wave evidence, not guessed permanently at the start.



11. Build the economics and TCO model

Evaluate modernization economics across the complete transition - current state, parallel-run period and target state - rather than comparing steady-state platform costs alone.

Current-state cost

- virtualization subscriptions/licenses;
- server hardware and refresh;
- storage;
- network;
- backup/DR;
- management tooling;
- security tooling;
- datacenter/colocation/facilities;
- support;
- people/operations;
- public cloud spend already associated with the estate.

Target-state run cost

- platform/software subscriptions;
- infrastructure or cloud consumption;
- storage and data protection;
- network/egress/connectivity;
- security and observability;
- managed-service charges;
- people/platform operations;
- support;
- lifecycle/upgrade activities.

Transition cost

- discovery and assessment;
- target platform build;
- migration tooling;
- migration engineering;
- application remediation;

- testing;
- training;
- professional services;
- parallel run of old and new estates;
- temporary capacity;
- decommissioning.

Five-year model

5-year modernization economics = transition cost + 5-year target run cost - avoided 5-year current-state cost

Also quantify non-cost outcomes:

- provisioning lead time;
- automation coverage;
- recovery confidence;
- platform consolidation;
- developer lead time;
- security policy consistency;
- reduction in manual operations.

Do not manufacture ROI from unverified productivity assumptions.

12. Capacity and infrastructure prerequisites

Before platform build, validate:

Compute

- CPU architecture and compatibility
- memory density
- NUMA / huge pages where required
- GPU/accelerator needs
- failure-domain sizing
- N+1 or required resilience headroom

Storage

- block/file/object requirements
- latency/IOPS/throughput
- snapshots/clones
- backup integration
- replication/DR
- shared storage versus local/SDS
- capacity growth
- CSI/platform compatibility where Kubernetes is used

Network

- IP/VLAN/subnet design
- east-west and north-south traffic
- MTU
- routing/BGP/EVPN requirements where relevant
- load balancing
- DNS/NTP
- firewall/security zones
- multi-network requirements
- DC/DR connectivity

Platform dependencies

- identity

- PKI/certificates
- registry/repositories
- automation
- monitoring/logging
- backup
- ITSM/CMDB
- security operations integration



13. Skills and operating model

Technology convergence without operational convergence creates a new silo.

Assess capability across:

- virtualization
- Linux/Windows
- Kubernetes
- networking
- storage
- automation/IaC
- GitOps
- security
- observability
- backup/DR
- SRE/platform engineering
- FinOps/capacity management

Operating-model evolution

Traditional pattern	Modern platform pattern
Console-driven administration	API/declarative automation
Ticket-based provisioning	Guardrailed self-service
Separate VM/container teams	Platform capability teams where appropriate
Manual configuration	IaC/GitOps
Reactive monitoring	SLO/observability-driven operations
Periodic policy checks	Continuous policy controls
Capacity by allocation	Capacity by demand, resilience and growth

The goal is not to erase infrastructure expertise. It is to apply that expertise through more automated and reusable platform capabilities.



14. Day-2 readiness

A platform is not modern because installation was automated.

Before production migration, answer:

- Who patches and upgrades it?
- How are capacity thresholds detected?
- How are certificates rotated?
- How is access reviewed?
- How are security policies enforced?
- How are backups tested?
- How is DR invoked and validated?
- How are failed nodes/storage/network components handled?
- What are platform SLOs?
- Who owns incidents at 2 AM?
- What vendor/community escalation paths exist?
- How is configuration drift controlled?
- How is cost/capacity reported?

If the organization cannot explain how the target platform will be operated during failure, the architecture is not finished.

15. Transformation roadmap

Phase 0 - Align

- agree modernization trigger and outcomes;
- establish executive sponsorship;
- define decision rights and program governance;
- establish financial baseline.

Phase 1 - Discover and decide

- inventory and dependency mapping;
- workload disposition;
- utilization analysis;
- execution-model decisions;
- placement decisions;
- target operating model;
- initial business case.

Exit: target platform strategy and migration candidates approved.

Phase 2 - Establish the foundation

- compute/network/storage;
- platform;
- IAM/security;
- observability;
- automation;
- backup/DR;
- operational processes;
- skills uplift.

Exit: production-ready platform acceptance.

Phase 3 - Pilot and industrialize

- representative pilots;
- validate tooling and patterns;
- measure migration throughput;
- refine complexity and effort model;
- establish migration factory.

Exit: repeatable migration patterns and evidence-based throughput.

Phase 4 - Migrate in waves

- application-group waves;
- remediation;
- migration/cutover;
- stabilization;
- legacy decommissioning.

Exit: target migration scope completed and legacy cost retired.

Phase 5 - Modernize selectively

- replatform/refactor applications where justified;
- expand developer self-service;
- automate operations;
- consolidate platform services;
- optimize placement and cost.

Modernization continues after infrastructure migration.



16. Decision gates for CXOs and architecture boards

Gate 1 - Is the problem understood?

- Is the trigger explicit?
- Is there a deadline or forcing event?
- Are expected outcomes measurable?

Gate 2 - Is the estate understood?

- Is inventory trustworthy?
- Are dependencies mapped?
- Is utilization known?
- Are business owners identified?

Gate 3 - Is platform strategy workload-driven?

- Have virtualization, Kubernetes, unified and mixed options been evaluated?
- Are workloads being forced to a predetermined destination?
- Is cloud placement evidence-based?

Gate 4 - Is the operating model credible?

- Are skills and ownership identified?
- Has build/buy/consume been decided by capability?
- Are Day-2 processes designed?

Gate 5 - Is the business case complete?

- Does it include transition/parallel-run cost?
- Are savings tied to actual retirement?
- Are assumptions documented?

Gate 6 - Is migration industrialized?

- Are representative pilots complete?
- Are wave patterns repeatable?
- Is throughput evidence-based?
- Is decommissioning part of completion?

17. Common failure modes

“Replace VMware first; modernize later”

Can be valid tactically, but may lock the next infrastructure cycle into the current operating model unless the long-term platform decision is made first.

“Everything must be containerized”

Creates transformation cost without guaranteed business value. Containerize where lifecycle, scale, resilience, delivery or portability benefits justify it.

“Everything should move to cloud”

Placement should follow workload characteristics and economics, not a blanket policy.

“Kubernetes will reduce operations automatically”

Kubernetes can enable automation and convergence, but it also requires mature platform operations.

“The PoC worked, therefore the platform is ready”

A booted VM or deployed container does not validate backup, DR, upgrades, observability, security, supportability or operating ownership.

“The migration business case assumes immediate savings”

Legacy cost is only removed when licenses, hardware, facilities or services can actually be retired.

“VM count is the migration plan”

Application dependencies and business cutovers, not VM objects, determine migration complexity.

18. Architect's final test

A credible Datacenter Modernization strategy should allow an executive or architecture board to answer:

- Why are we modernizing?
- Which workloads should be retained, retired, replaced, rehosted, replatformed or refactored?
- Which workloads belong on VMs, containers, bare metal, SaaS or managed services?
- Do we need virtualization, Kubernetes, a unified platform or a mixed estate?
- Which workloads belong on-premises, in public cloud or at the edge?
- What should we build, buy or consume?
- What capabilities must the target platform provide?
- What are the infrastructure and organizational prerequisites?
- How will workloads be migrated safely?
- What drives effort and timeline?
- What is the complete five-year economic model, including transition cost?
- Who will operate the platform on Day 2?
- What evidence tells us the program is succeeding?

If these questions can be answered before product selection, the resulting architecture is far more likely to be a modernization strategy rather than a technology substitution exercise.

Closing position

The target datacenter should emerge from workload, operating-model and economic decisions - not the other way around.

The most important choice may not be which hypervisor or Kubernetes distribution to buy. It may be deciding which workloads should remain virtual machines, which should become cloud-native, which should move to a service, where each should run, and how much platform responsibility the enterprise should continue to own.

That is the foundation for an open, hybrid and evolvable datacenter.

Modernize by workload and operating-model decision, not by predetermined technology destination.